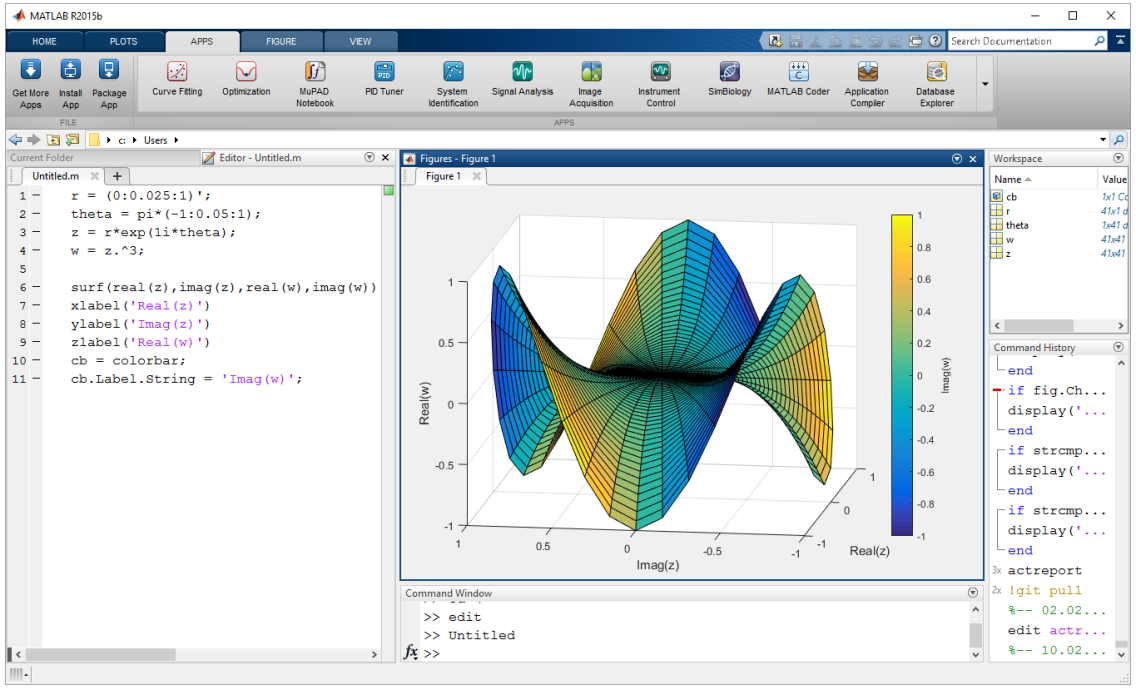
Numerical Methods

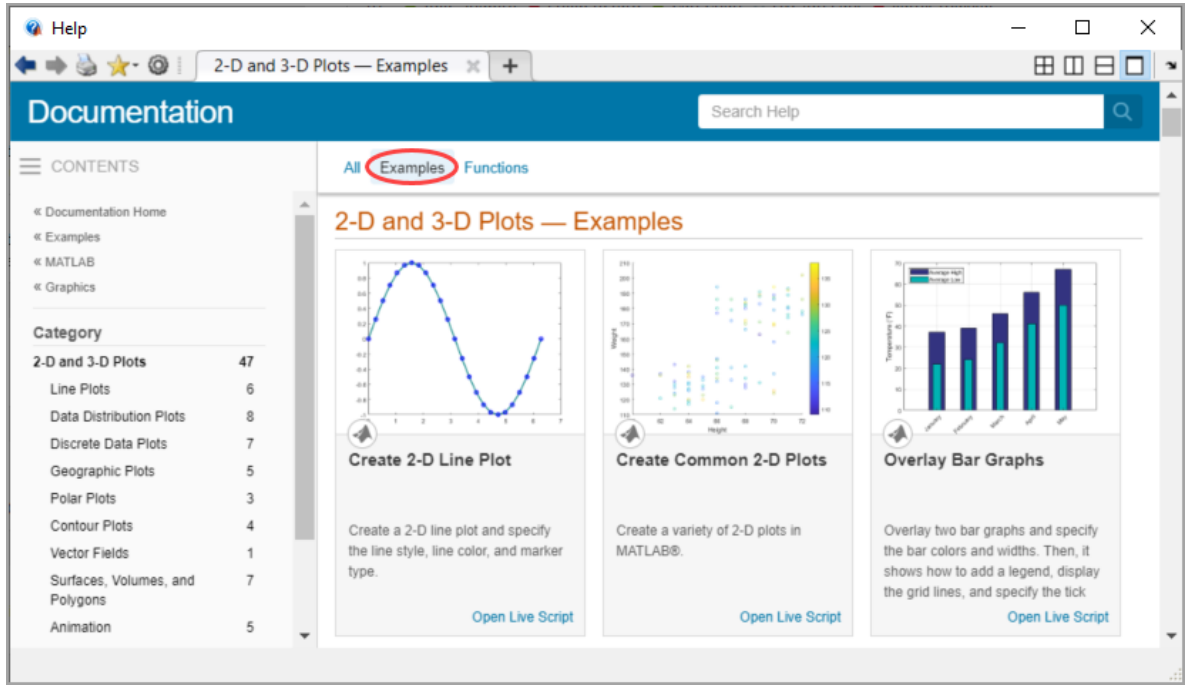
Dr. Thai Minh Quan

Tel: 081.555.9669

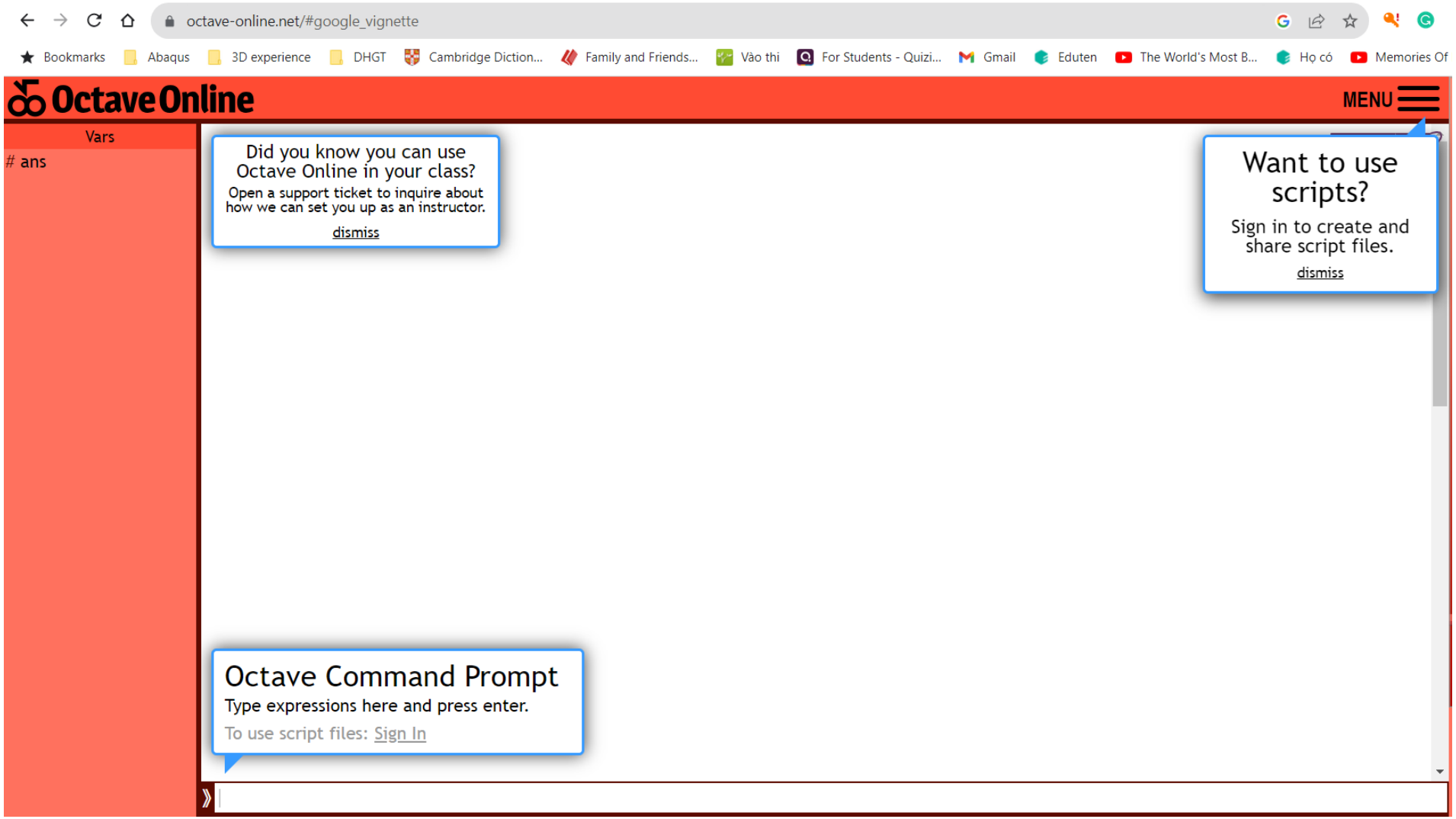
Thai-minh.quan@usth.edu.vn

Matlab/Octave introduction





Matlab/Octave introduction



Matlab introduction



- Stands for MATrix LABoratory
- Interpreted language
- Scientific programming environment
- Very good tool for the manipulation of matrices
- Great visualisation capabilities
- Loads of built-in functions
- Easy to learn and simple to use

Matlab introduction

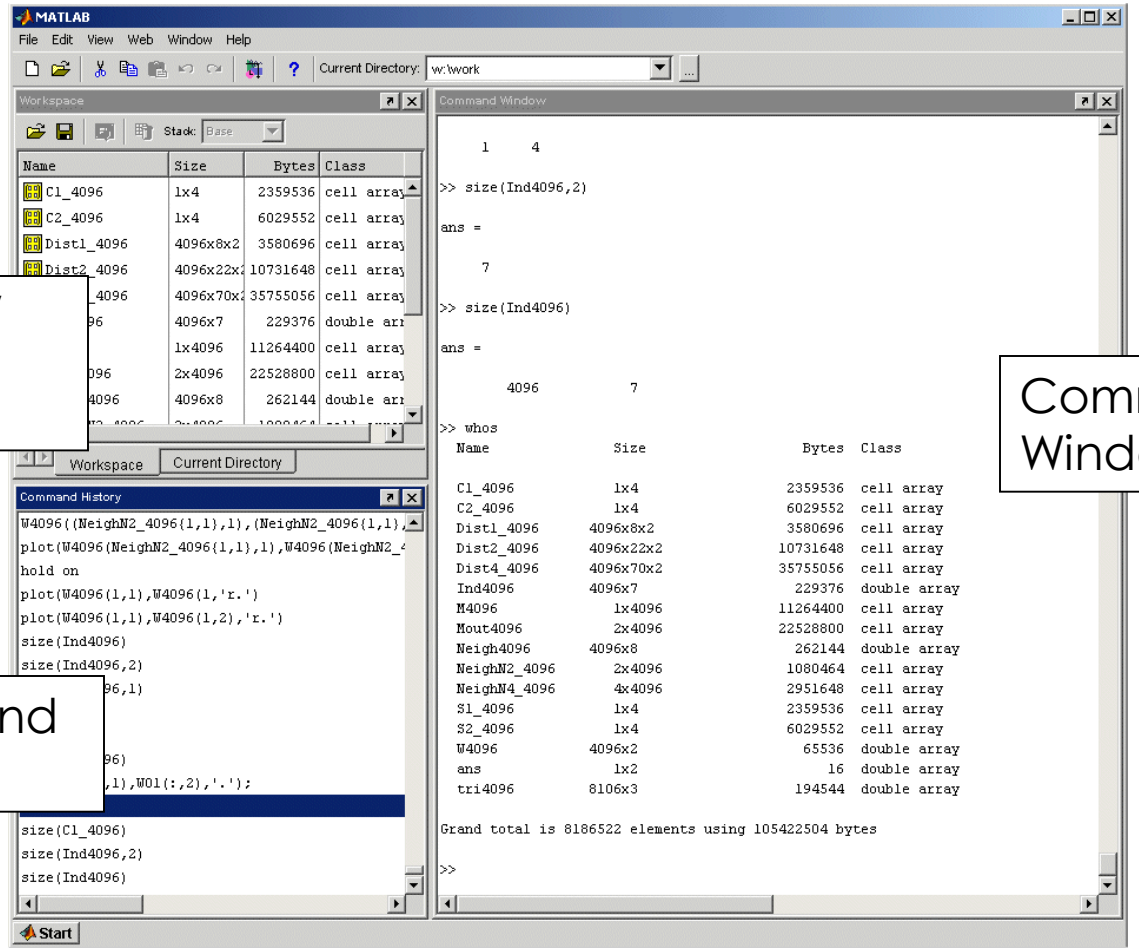
MATLAB software

- A convenient environment for performing many types of calculations
- A very nice tool to implement numerical methods

MATLAB uses three primary windows:

- Command window. Used to enter commands and data.
- Graphics window. Used to display plots and graphs.
- Edit window. Used to create and edit M-files.

Matlab Desktop



Workspace / Current Directory

Command History

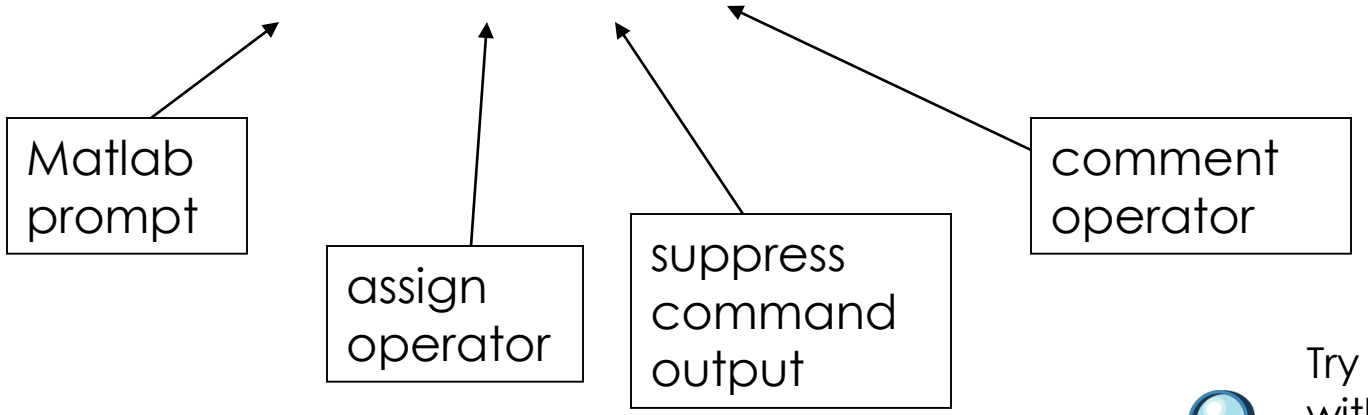
Command Window

Explore the Matlab Desktop

Variables

- Don't have to declare type
- Don't even have to initialise
- Just assign in command window

```
>>
>> a=12; % variable a is assigned 12
```





Try the same line without the semicolon and comments

Variables (continued …)

- View variable contents by simply typing the variable name at the command prompt

```
>> a
a =
        12
>>
>> a*2
a =
        24
>>
```

Matlab introduction

Mathematical operations

- Type 2+3 after the >> prompt, followed by **Enter** (press the **Enter** key) as indicated by <**Enter**>:
  >> 2+3 < **Enter**>
- Commands are only carried out when you enter them. The answer in this case is, of course, 5. Next try
  >> 3–2 < **Enter**>
  >> 2*3 < **Enter**>
  >> 1/2 < **Enter**>
  >> 2^3 < **Enter**>
  >> 2\1 < **Enter**>

^	Exponentiation
–	Negation
* /	Multiplication and division
\	Left division[2]
+ –	Addition and subtraction

Workspace

- The workspace is Matlab's memory
- Can manipulate variables stored in the workspace

```
>> b=10;
>> c=a+b
c =
    22
>>
```

Workspace (continued …)

- Display contents of workspace

```
>> whos
 Name      Size                Bytes  Class
  a        1x1                     8  double array
  b        1x1                     8  double array
  c        1x1                     8  double array
Grand total is 3 elements using 24 bytes
>>
```

- **Delete variable(s) from workspace**

```
>> clc
>> clear all; % delete a and b from workspace
>> close all; % delete all variables from workspace
```

Matlab help commands

- help

>> help sum % displays documentation for the function sum

>> lookfor convert % displays functions with convert in the first help line

- Start Matlab help documentation

>> helpdesk

Matlab introduction

- **Vector**
  - ✓ a = [1 2 3 4 5] or >> a = 1:5
    and
  - ✓ a = [1 ; 2 ; 3 ; 4 ; 5] or a = [1 2 3 4 5]'

- **Matrix**
  - ✓ A = [1 2 3; 4 5 6; 7 8 9]
  - ✓ z = zeros(2,4)
  - ✓ x = ones(2,3)

$$z = zeros(2, 4)$$

$$z = \begin{matrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix}$$

$$x = ones(2, 3)$$

$$x = \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

Matlab introduction

- **Matrix**
  - ✓ `M=eye(3)`

```
M =

     1     0     0
     0     1     0
     0     0     1
```

```
M=rand(3)

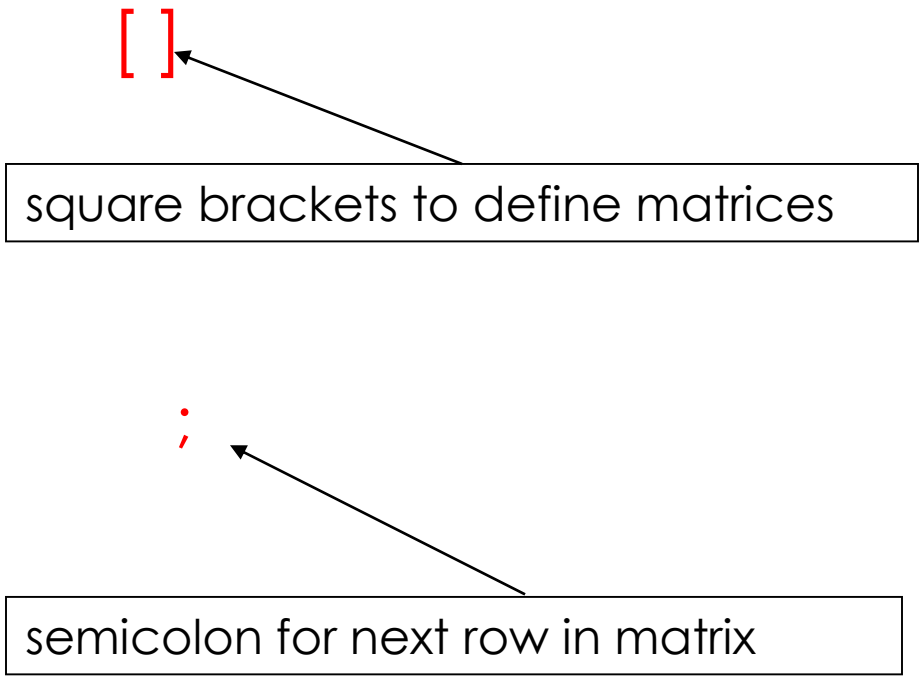
M =

    0.687971   0.207907   0.359514
    0.810225   0.967850   0.541394
    0.716802   0.802503   0.063585
```

Matrices

- Don't need to initialise type, or dimensions

```
A = [3 2 1; 5 1 0; 2 1 7]
A =
     3     2     1
     5     1     0
     2     1     7
```



Manipulating Matrices

A =

3	2	1
5	1	0
2	1	7

- Access elements of a matrix

A(1,2)

ans=

2



- Remember Matrix(row,column)
- Naming convention Matrix variables start with a capital letter while vectors or scalar variables start with a simple letter

The : operator

- VERY important operator in Matlab
- Means ‘to’

```
>> 1:10
ans =
     1     2     3     4     5     6     7     8     9    10
>> 1:2:10
ans =
     1     3     5     7     9
```

🔎 Try the following

```
>> x=0:pi/12:2*pi;
>> y=sin(x)
```

The : operator

- **Defines a *range*** in expressions like

```
indices = 1:5
```
Returns row vector 1,2,3,4,5

```
steps = 1:3:61
```
Returns row vector 1,4,7,...,61

```
t = 0:0.01:1
```
Returns vector 0,0.01,0.02,...,1

start increment stop

The : operator and matrices

```
A(3,2:3)
ans =
     1     7
A(:,2)
ans =
     2
     1
     1
```

```
A =
     3     2     1
     5     1     0
     2     1     7
```



What'll happen if you type A(:,:) ?

Manipulating Matrices

```
A =
     3     2     1
     5     1     0
     2     1     7
```

```
B =
     1     3     1
     4     9     5
     2     7     2
```

```
>> A '      % transpose
>> B*A      % matrix multiplication
>> B.*A     % element by element multiplication
>> B/A      % matrix division
>> B./A     % element by element division
>> [B A]    % Join matrices (horizontally)
>> [B; A]   % Join matrices (vertically)
```



Enter matrix B into the Matlab workspace



Create matrices A and B and try out the the matrix operators in this slide

Manipulating Matrices

```
M=[1 2 3;1 2 3;1 2 3]
```

```
M =

     1     2     3
     1     2     3
     1     2     3
```

```
M.^2
```

```
ans =

     1     4     9
     1     4     9
     1     4     9
```

Manipulating Matrices

```
a=rand(3,1)      a =

                   0.16028
                   0.80442
                   0.79403
```

```
b=rand(3,1)      b =

                   0.722985
                   0.098011
                   0.415948
```

```
c=[a b]          c =

                   0.160277   0.722985
                   0.804421   0.098011
                   0.794033   0.415948
```

Built-in functions in Matlab/Octave

```
a=[1 2 3; 4 5 6; 6 7 8; 9 10 11]
```

```
a =

    1    2    3
    4    5    6
    6    7    8
    9   10   11
```

```
s=size(a)
```

```
s =

    4    3
```

Built-in functions in Matlab/Octave

```
a=pi/3;
sin(a)
```

```
ans = 0.86603
```

```
a=rand(5,1)
```

```
a =

   0.46179
   0.66944
   0.32697
   0.46256
   0.13318
```

```
m=max(a)
```

```
m = 0.66944
```

Built-in functions in Matlab/Octave

```
a=rand(5,1)
```

```
a =

   0.670587
   0.855858
   0.984794
   0.568653
   0.090375
```

```
m=mean(a)
```

```
m = 0.63405
```

Visualisation - plotting data

Plotting

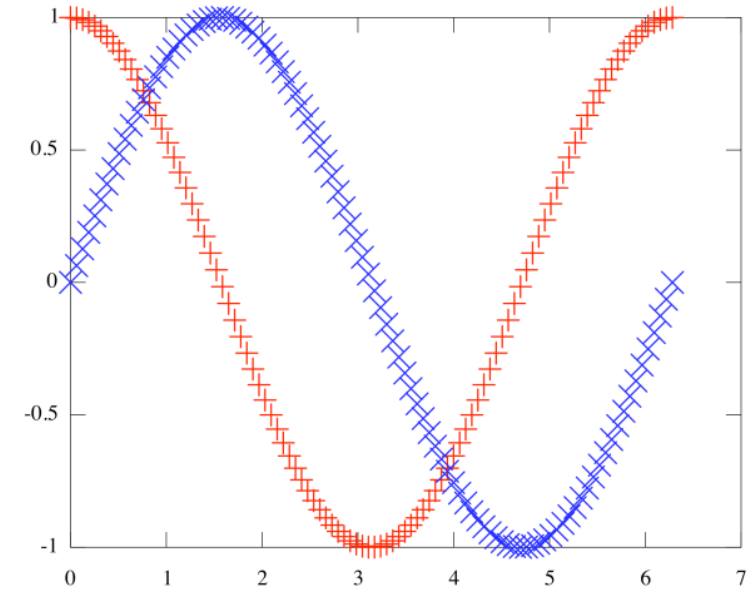
Choosing Symbols and Colors

- In `plot(x,cos(x),'r+')` the format expression `'r+'` means *red cross.*
- There are a **number of line styles and colors,** see `help plot`.

Example:

```
octave:1> x = linspace(0,2*pi,100);
octave:2> plot(x,cos(x),'r+',x,sin(x),'bx')
```

produces this plot:

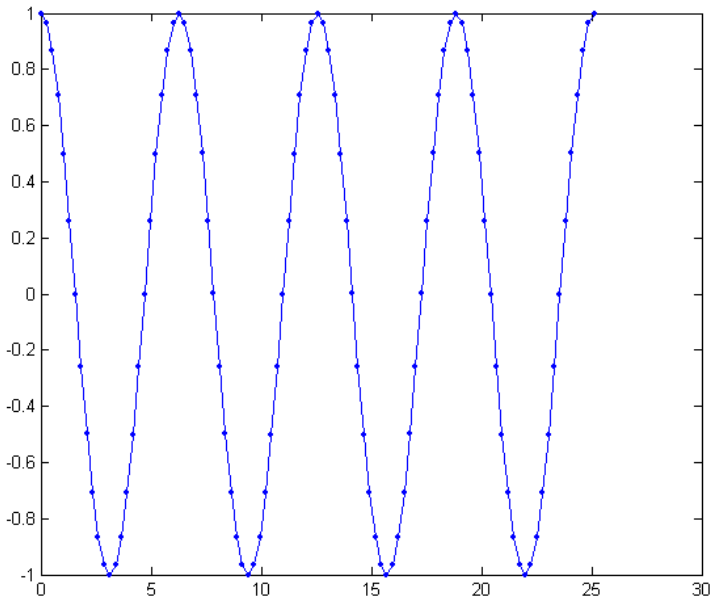


`plot(x,cos(x),'r+',x,sin(x),'bx');`

Visualisation - plotting data

```
>> figure   % create new figure
>> t=0:pi/12:8*pi;
>> y=cos(t);
>> plot(t,y,'b.-')
```

Plot style



Investigate the function
>> y=A*cos(w*t+phi);
for different values of phi (eg: 0, pi/4, pi/3, pi/2), w (eg: 1, 2, 3, 4) and A (eg: 1, 0.5, 2). Use the **hold on** Matlab command to display your plots in the same figure. Remember to type **hold off** to go back to normal plotting mode. Try using different plot styles (**help plot**)

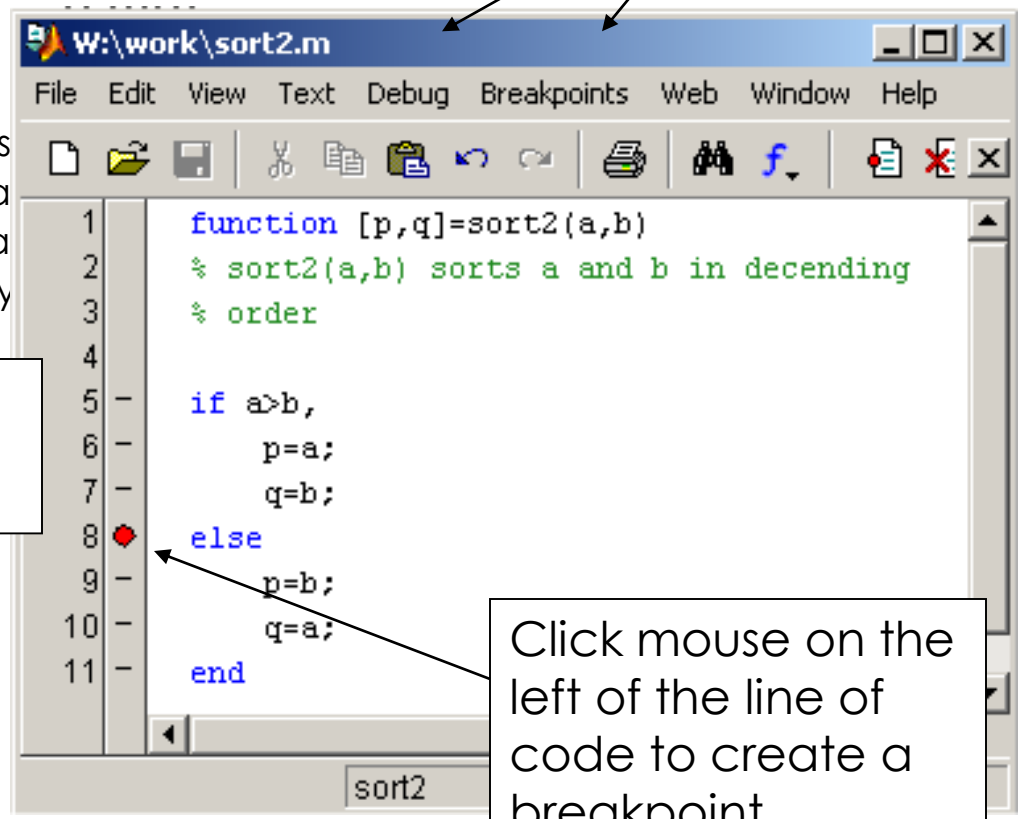
A = amplitude
phi = phase
w = angular frequency = 2*pi*frequency

Debugging

- Set breakpoints to stop the execution of code

```
>> [i j]=sort2(2,4)
K>>
K>> whos
 Name      Size                Bytes  Class
  a         1x1                     8  double a
  b         1x1                     8  double a
Grand total is 2 elements using 16 by
K>> a
a =
     2
K>> return
i =

     4
j =
     2
```



Matlab introduction

- Some commonly used tools and commands:
  - ↑ (up arrow) returns last command input, can be repeated
  - clc – clears the screen
  - clear all – clears variables
  - close all – closes figures

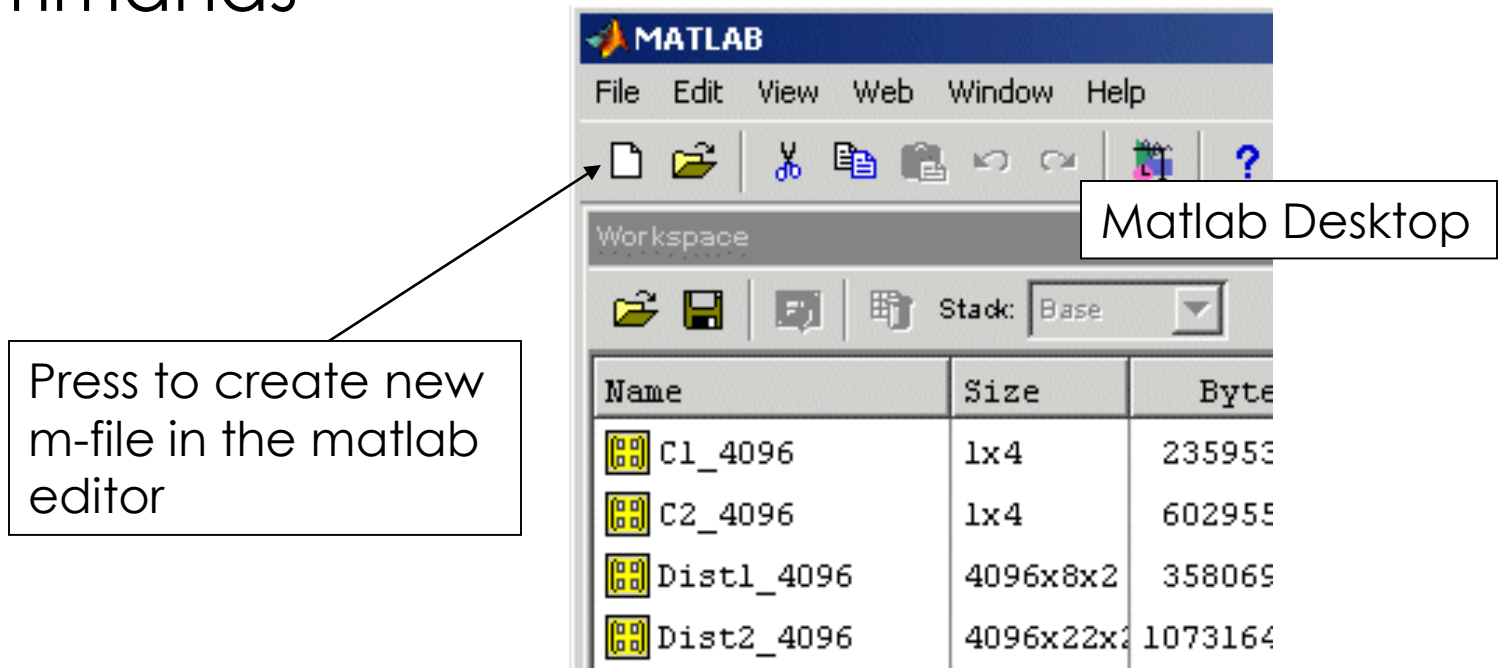
M-FILES

- An M-file consists of a series of statements that can be run all at once.
- M-files are stored with a .m extension
- M-files come in two flavors: script files and function files
  - A script file is merely a series of MATLAB commands that are saved on a file
  - Function files are M-files that start with the word *function*. In contrast to script files, they can accept input arguments and return outputs.

    languages such as *Fortran*, *Visual Basic* or *C*.

Scripts

- Matlab editor
- Use scripts to execute a series of Matlab commands



User-defined functions

```
function [output1, output2, ...]=function_name(input1, input2, ...)
   body of the function
```

```
## A meaningless function that takes a number as input, multiplies it by
## three
## Usage result=times_three(input)

function result=times_three(input)

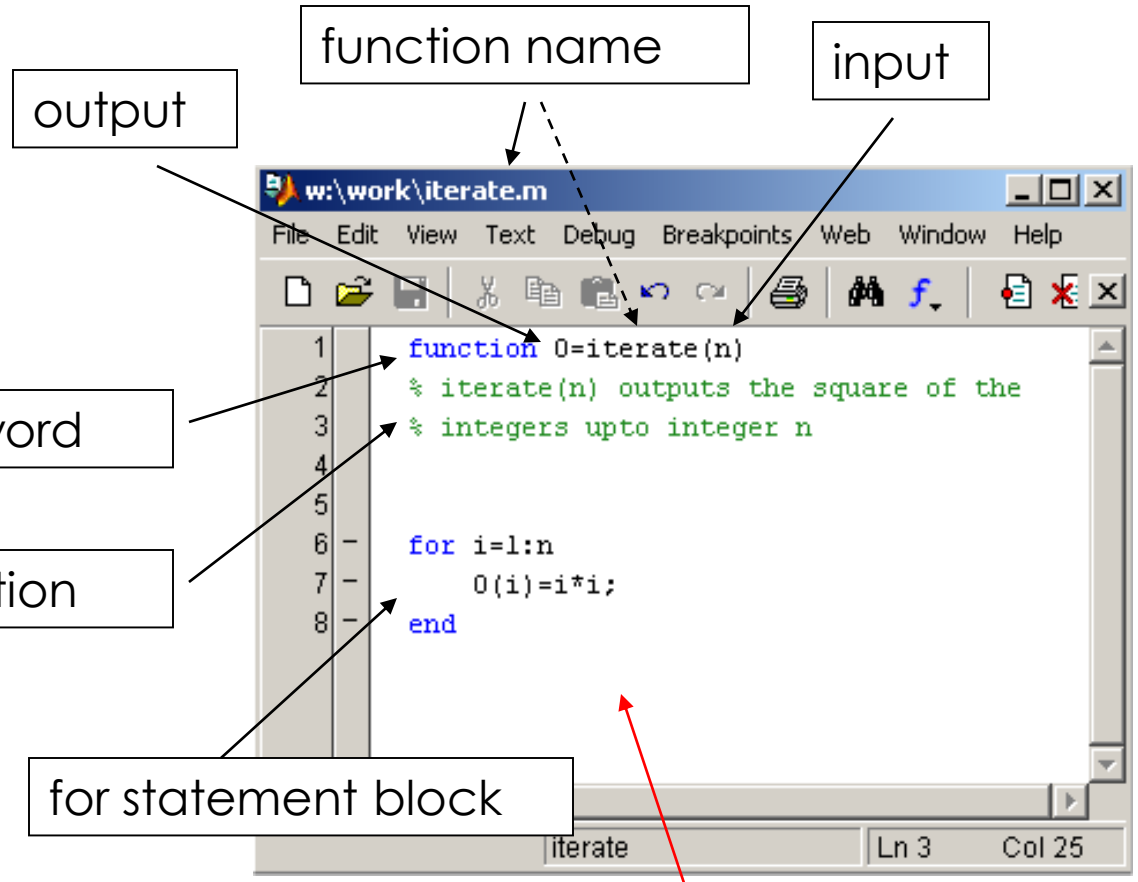
  result = input*3;
```

```
a=times_three(3)

a = 9
```

Functions (continued)

```
>> I=iterate(5)
I =
     1     4     9    16    25
```

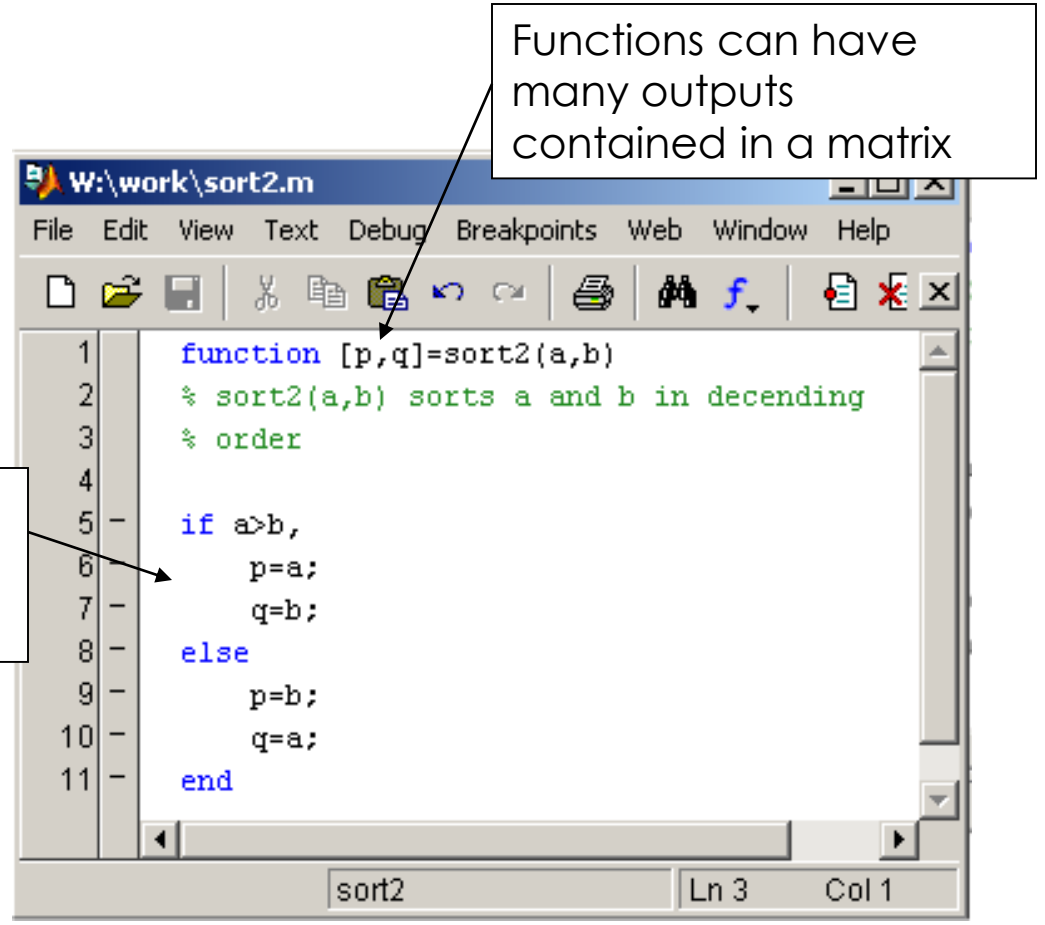


Access the comments of your Matlab functions
>> help iterate

Make sure you save changes to the m-file before you call the function!

Functions (continued)

```
>> [i j]=sort2(2,4)
i =
     4
j =
     2
>>
```



Remember to use the Matlab help command for syntax
>> help if

Function files

- The syntax for the function file can be represented generally as

function outvar = funcname(arglist)

% helpcomments

statements

outvar = value;

The M-file should be saved as funcname.m

Function files

```
function y = myfunc(x)
    y = 2*x.^2 - 3*x + 1;
```

All function programs, the essential elements are:

- Begin with the word function.
- There is an input and an output.
- The output, name of the function and the input must appear in the first line.
- The body of the program must assign a value to the output.

```
x = -2:.1:2; ..
y = myfunc(x);
plot(x,y)
```

Script file

Function files

- **Example**

function [mean,stdev] = stat(x)
n = length(x);
mean = sum(x)/n;

```
clc
close all
clear all
%%
x = 1:5

[mean] = stat(x)
```

The if...else Structure

- **Structure**

if condition
Statements 1
else
Statements 2
end

```
% Example
attendance = 5;
grade_average = 15;
if ((attendance >= 6) && (grade_average >= 10))
   pass = 1
else
   fail = 1
end;
```

The for loop

- **Structure**

for index = j: k
statements
end

```
% Example
a = 0;
for i=1:10
    a=a+i;
end
```

The While loop

- **Structure**

while condition
statements
end

```
% Example
n = 10;
f = n;
while n > 1
    n = n-1;
    f = f+n;
end
```

Case study

1. Write a program to solve linear equation in matlab

$$ax + b = 0$$

Case study

- Function file

```
function x = linear_eq(a,b)
x=-b/a;
```

- Script file

```
clc
clear all
close all
%%
a = 2;
b = 6;
if a == 0
    fprintf('Coefficient of "a" must not be zero!');
else
    x = linear_eq(a,b)
end
```

1. Make Sure the Function Name Matches the File Name

You establish the name for a function when you write its function definition line. This name should always match the name of the file you save it to. For example, if you create a function named curveplot,

function curveplot(xVal, yVal)

then you should name the file containing that function curveplot.m

If not: error!

"*Undefined function or variable curveplot*"

Matlab-Bisection method

- INPUT: Function f, endpoint values a, b, tolerance TOL, maximum iterations NMAX

CONDITIONS: a < b, either f(a) < 0 and f(b) > 0 or f(a) > 0 and f(b) < 0

- OUTPUT: value which differs from a root of f(x)=0 by less than TOL

```
N ← 1
While N ≤ NMAX # limit iterations to prevent infinite loop
        c ← (a + b)/2 # new midpoint
        If f(c) = 0 or (b – a)/2 < TOL then # solution found
                Output(c)
                Stop
        EndIf
       N ← N + 1 # increment step counter
       If sign(f(c)) = sign(f(a)) then a ← c else b ← c # new interval
EndWhile
Output("Method failed.") # max number of steps exceeded
```

Algorithm for Newton-Raphson

1. A plotting routine should be included in the program.
2. At the end of the computation, the final root estimate should always be substituted into the original function to compute whether the result is close to zero. This check partially guards against those cases where slow or oscillating convergence may lead to a small value of ε_a while the solution is still far from a root.
3. The program should always include an upper limit on the number of iterations to guard against oscillating, slowly convergent, or divergent solutions that could persist interminably.
4. The program should alert the user and take account of the possibility that $f'(x)$ might equal zero at any time during the computation.

Algorithm for Newton-Raphson

input $x,\ M$

$k \leftarrow 0$

$y \leftarrow f(x)$

print $k,\ x,\ y$

for $k = 1,\ 2, \ldots, M$

$x \leftarrow x - \frac{y}{f'(x)}$

$y \leftarrow f(x)$

print $k,\ x,\ y$

end

$f(x) = e^{-x} - x$

%% **Newton Raphson**

$$f(x) = e^{-x} - x$$

```
x = 0;
x_i = 1;
iter = 0;
eps_s = 10^-8
while abs(x_i-x)> eps_s
    x_i = x;
    x = x - (exp(-x) - x)/(-exp(-x) - 1);
    iter = iter + 1;
    fprintf('Iteration %d: x=%.20f, err=%.20f\n', iter, x, abs((x_i-x)/x)*100);
    pause;
end
```